



UNIVERSIDADE ESTADUAL DA PARAÍBA
BACHARELADO EM CIÊNCIAS DA COMPUTAÇÃO

ARTHUR MARQUES ARAUJO

GUILHERME SILVA HENRIQUES DE SOUSA

LUIZ JOSÉ MENDONÇA DUARTE

MANOEL BARROS DA CRUZ NETO

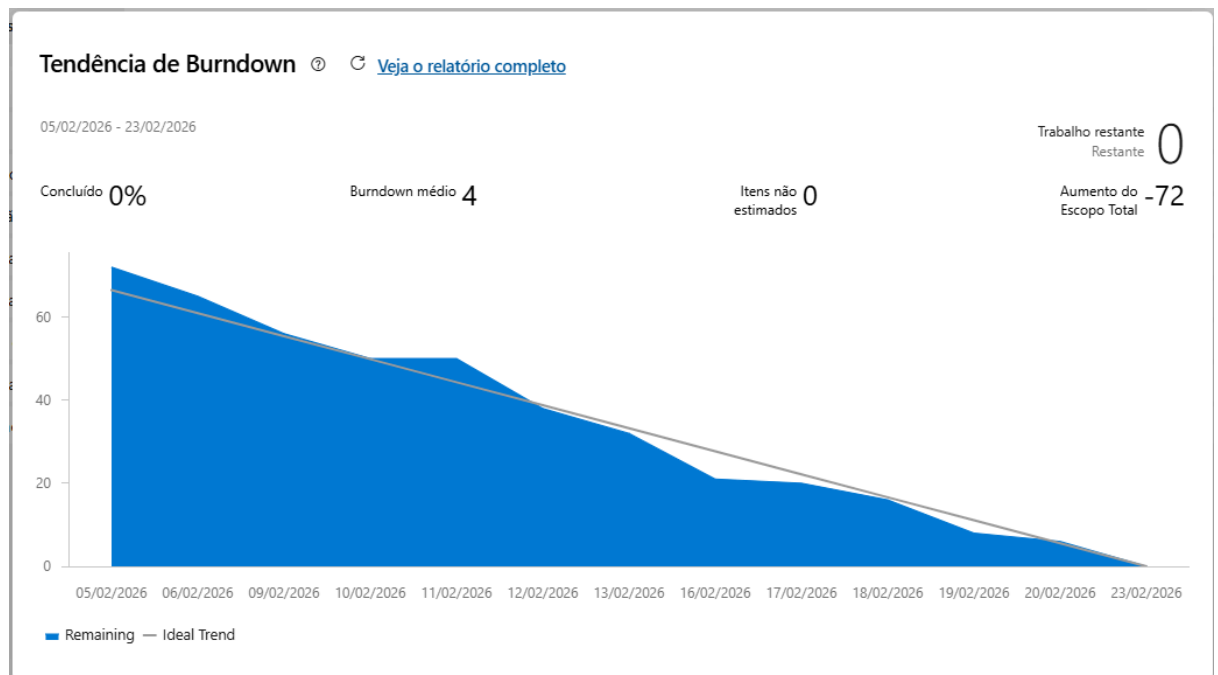
RELEASE 3

CAMPINA GRANDE

2026

SPRINT 5

GRÁFICO



RESUMO

A Sprint 5 teve como foco a expansão total do sistema UberPB para o domínio de delivery (modelo UberEats). O objetivo desta iteração foi entregar o fluxo completo (End-to-End) de pedidos de comida, englobando o cadastro dos novos atores (RF01 e RF02), a gestão de cardápios e pedidos (RF19 e RF20), o fluxo de preparo pelo restaurante (RF21), a atribuição automatizada de entregadores (RF22) e a finalização com pagamento (RF23).

ORGANIZAÇÃO DA EQUIPE

O processo de desenvolvimento desta primeira etapa da sprint seguiu a seguinte divisão estrutural:

- **Manoel Barros (Desenvolvedor):** Responsável por encabeçar a modelagem de dados, as regras de validação, a persistência JSON e a interface via linha de comando (CLI) relativas às entidades Entregador e Restaurante.
- **Arthur Marques (Desenvolvedor):** Responsável pela gestão de oferta do delivery, atuando na modelagem e exibição estruturada de cardápios e na lógica de listagem de restaurantes disponíveis. Integrou as regras de negócio de disponibilidade desde a camada de persistência até a interface de usuário.
- **Guilherme Silva (Desenvolvedor):** Encarregado da implementação central do ciclo de vida dos pedidos e do algoritmo de atribuição de entregadores. Construiu as entidades fundamentais, desenvolveu a lógica de transição de status e adaptou os menus interativos do Restaurante e do Entregador.

- **Luiz Duarte (Desenvolvedor e Tester):** Responsável por implementar as lógicas de temporalidade dos pedidos, desenvolvendo a infraestrutura para entregas agendadas e imediatas. Além disso, desenvolveu a suíte completa de testes unitários, assegurando a robustez e a cobertura dos novos requisitos incorporados na sprint.

IMPLEMENTAÇÕES

1. Expansão do Domínio para Delivery (RF01 e RF02)

Responsável: Manoel Barros

Status:  Concluído

Nesta etapa inicial da sprint, o escopo do sistema foi expandido para integrar os perfis de Entregadores e Restaurantes à base já existente, atendendo aos requisitos de cadastro (RF01) e validação de documentos e veículos (RF02).

1.1. Modelagem de Dados (Camada Model)

A arquitetura orientada a objetos foi respeitada utilizando o conceito de herança:

- **Classe Entregador:** Estende a classe base `User`. Foram adicionados atributos específicos para a operação de delivery, como `cnh`, `tipoVeiculo` (restrito a MOTO ou BICICLETA), status de `disponivel` e `localizacaoAtual`. A entidade também herdou a estrutura de gestão de avaliações.
- **Classe Restaurante:** Estende a classe base `User`. Recebeu atributos operacionais próprios, como `cnpj`, `razaoSocial`, endereço completo e um status booleano `aberto` para gerenciar o horário de funcionamento.

1.2. Validações e Regras de Negócio (Camada Helpers e Services)

A integridade dos dados foi garantida através de validações estritas antes da persistência:

- **Validação de CNPJ:** Implementado um novo validador na classe `ValidadoresCadastro`, exigindo exatamente 14 dígitos numéricos.
- **Validação de Veículos:** Adicionada a regra de negócio que restringe os veículos de entregadores exclusivamente aos tipos "MOTO" ou "BICICLETA".
- **Services Dedicados:** Criadas as classes `EntregadorService` e `RestauranteService` para atuar como intermediárias entre a interface e a camada de dados, isolando as validações.

1.3. Persistência e Concorrência (Camada Repository)

O padrão *Repository Pattern* baseado em arquivos JSON foi expandido mantendo a segurança contra concorrência (*Thread-Safety*):

- **Novos Repositórios:** Criadas as classes `EntregadorRepositoryJSON` e `RestauranteRepositoryJSON`, herdando da `BaseRepository`.
- **Gerenciamento do Jackson:** A classe abstrata `BaseRepository` foi atualizada no método `loadAll()` com mapeamento explícito (via `switch-case` com `TypeReference`) para as entidades "entregadores" e "restaurantes". Isso garante a desserialização correta e previne erros de `ClassCastException`.
- **Integração com Facade:** O `DatabaseManager` foi atualizado para instanciar os novos repositórios e delegar as operações (ex: `saveEntregador`, `findRestauranteById`).

1.4. Interface com o Usuário (Camada CLI)

A experiência via linha de comando foi aprimorada para suportar os novos fluxos sem quebrar o padrão de navegação já estabelecido:

- **Menu Principal:** Adicionadas as opções de "Cadastrar perfil de Entregador" e "Cadastrar perfil de Restaurante", reaproveitando os dados da sessão do usuário logado.
- **Menu Entregador:** Interface própria que permite ao entregador visualizar sua avaliação média, atualizar sua localização atual via `LocalizacaoService` e alternar seu status de disponibilidade (Online/Offline).
- **Menu Restaurante:** Interface enxuta que permite ao estabelecimento alternar seu status de operação (Aberto/Fechado) para começar a receber pedidos.

2.1 Listagem de Restaurantes Disponíveis (RNF19)

Responsável: Arthur Marques Araújo

Status: Concluído

A funcionalidade de listagem de restaurantes disponíveis foi implementada com o objetivo de permitir ao usuário visualizar apenas os estabelecimentos que se encontram abertos no sistema.

2.1.1 Modelagem de Dados (Camada Model)

A classe `Restaurante`, já existente no sistema, é responsável por representar os restaurantes cadastrados. Essa classe contém atributos como `cnpj`, `razaoSocial`, `endereco` e `statusAberto`, sendo este último utilizado como critério para determinar a disponibilidade do restaurante.

2.1.2 Persistência e Regras de Negócio (Camada Repository)

Na camada de persistência, foi adicionado o método `findDisponiveis()` à classe `RestauranteRepositoryJSON`, permitindo a recuperação exclusiva dos restaurantes com status de aberto.

Além disso, foi implementado o método `findRestaurantesDisponiveis()` na classe `DatabaseManager`, com o objetivo de disponibilizar essa funcionalidade para as demais camadas do sistema, promovendo a integração entre os componentes.

2.1.3 Interface com o Usuário (Camada CLI)

Na camada de interface em linha de comando (CLI), especificamente na classe MenuPassageiroCLI, foi incluída uma nova opção para listagem de restaurantes disponíveis. A funcionalidade exibe informações como nome, endereço, avaliação e CNPJ do restaurante.

Caso não haja restaurantes disponíveis, o sistema apresenta uma mensagem informando a indisponibilidade no momento.

2.2 Modelagem e Exibição de Cardápio (RNF20)

Responsável: Arthur Marques Araújo

Status: Concluído

A funcionalidade de modelagem e exibição de cardápio foi implementada com a finalidade de permitir ao usuário visualizar os itens ofertados pelos restaurantes disponíveis, bem como informações complementares relacionadas ao serviço.

2.2.1 Modelagem de Dados (Camada Model)

Foram criadas as seguintes classes:

- Item: responsável por representar os itens do cardápio, contendo atributos como nome, preço e descrição;
- Cardapio: responsável por representar o conjunto de itens ofertados por um restaurante, contendo uma lista de itens, além de informações como taxa de entrega e tempo estimado de entrega.

Adicionalmente, a classe Restaurante foi atualizada para incluir o atributo cardapio, estabelecendo a associação entre restaurante e seus respectivos itens ofertados.

2.2.2 Persistência e Regras de Negócio (Camada Repository)

O cardápio foi associado diretamente ao restaurante na camada de persistência, permitindo sua manipulação e exibição de forma integrada ao fluxo do sistema.

2.2.3 Interface com o Usuário (Camada CLI)

Na classe MenuPassageiroCLI, foi adicionada uma opção para visualização do cardápio de um restaurante disponível. A funcionalidade exibe:

- Lista de itens;
- Preços;
- Taxa de entrega;
- Tempo estimado de entrega.

Caso o restaurante selecionado não possua cardápio cadastrado, o sistema apresenta uma mensagem informativa ao usuário.

3 Implementação do Fluxo de Pedido – RF21 e RF22

Responsável: Guilherme

Status: Concluído

Nesta etapa da Sprint 5 foram implementados os requisitos RF21 (fluxo de preparo pelo restaurante) e RF22 (atribuição de entregador), incluindo a criação das classes Pedido, ItemPedido e PedidoCLI, além de alterações estruturais nos menus de Restaurante e Entregador.

3.1 Classe Pedido (Camada Model)

A classe Pedido foi criada para representar o pedido de delivery dentro do sistema.

Principais atributos implementados:

- id
- clienteId
- restauranteId
- entregadorId
- lista de ItemPedido
- status
- valorTotal

Foi implementado o controle de estados do pedido, permitindo as seguintes transições relacionadas aos requisitos:

- AGUARDANDO_RESTAURANTE
- EM_PREPARO
- AGUARDANDO_ENTREGADOR
- EM_ENTREGA

Também foram implementados métodos para:

- Alteração controlada de status
- Associação de entregador ao pedido
- Cálculo automático do valor total com base nos itens

A classe passou a ser a entidade central do fluxo de delivery, sendo responsável por garantir a consistência das transições de estado.

3.2 Classe ItemPedido

A classe ItemPedido foi criada para representar cada item selecionado dentro de um pedido.

Principais atributos:

- Item item
- int quantidade
- double subtotal

O subtotal é calculado multiplicando o preço do item pela quantidade escolhida.

Essa separação garante:

- Organização dos itens dentro do pedido
- Cálculo correto do valor total
- Independência entre o item do cardápio e sua instância no pedido

A classe também contribui para manter a responsabilidade de cálculo descentralizada, tornando o modelo mais organizado.

3.3 Classe PedidoCLI (Camada CLI)

Foi criada a classe PedidoCLI para gerenciar as interações relacionadas ao pedido na interface de linha de comando.

Responsabilidades implementadas:

- Criação de pedido pelo cliente
- Exibição de pedidos
- Integração com restaurante e entregador
- Atualização e exibição de status

Essa classe atua como intermediária entre o usuário e a camada de modelo, garantindo separação de responsabilidades e organização da lógica de fluxo.

3.4 Alterações no Menu Restaurante

O menu do restaurante foi modificado para permitir o gerenciamento de pedidos recebidos.

Funcionalidades adicionadas:

- Visualização de pedidos associados ao restaurante
- Seleção individual de pedido
- Opção de aceitar pedido
- Opção de finalizar preparo

Regras implementadas:

- Pedido só pode ser aceito se estiver em AGUARDANDO_RESTAURANTE
- Pedido só pode finalizar preparo se estiver em EM_PREPARO

Ao finalizar o preparo, o pedido passa automaticamente para AGUARDANDO_ENTREGADOR, tornando-se disponível para entrega.

3.5 Alterações no Menu Entregador (RF22)

O menu do entregador foi alterado para permitir a visualização e aceitação de entregas.

Funcionalidades adicionadas:

- Listagem de pedidos aguardando entregador

- Opção de aceitar entrega
- Atualização do status para EM_ENTREGA
- Atualização automática da disponibilidade do entregador

Regras implementadas:

- Apenas entregadores disponíveis podem aceitar pedidos
- Um entregador não pode assumir múltiplas entregas simultaneamente

Quando o entregador aceita, o pedido recebe o entregadorId e o status é atualizado para EM_ENTREGA.

3.6 Testes Unitários

Foi desenvolvido teste unitário para a classe Pedido utilizando JUnit, com o objetivo de validar o funcionamento correto da lógica implementada.

Os testes contemplaram:

- Criação de pedido
- Adição de ItemPedido
- Cálculo correto do valor total
- Transições válidas de status
- Associação de entregador ao pedido

4. Criação de Tipos de Pedidos

Responsável: Luiz José

Status: Concluído

Foi implementado os tipos de Entrega como Enums, possibilitando que o usuário consiga fazer um pedido agendado ou de forma imediata, seguindo as especificações do RF23.

4.1 Modelagem de Dados (Camada Model)

Foram adaptadas as classes:

- **Cardápio:** Implementada a funcionalidade de `removerItem()`.
- **Pedido:** Adaptação dos atributos, para criação do `TipoEntrega` que contém as opções de entrega (agendada e imediata) e da variável para colocar a data de agendamento. Além, da função de agendar no caso de entrega agendada.

4.2 Interface com o Usuário (Camada CLI)

Foi adaptadas a classe:

- **PedidoCLI:** Adicionada a opção de agendamento no CLI, juntamente com a parte de agendar.

5. Testes Funcionais

Responsável: Luiz José

Status: Concluído

Foi implementado os testes referentes ao requisitos implementados para a Sprint 5, RF01 e RF02, e RF 19 até RF23, utilizando **JUnit**.

5.1 Testes (Camada Test)

Foram criados testes para os requisitos da Sprint, e testes para completar a cobertura do sistema como um todo:

- **CardápioTest:** Testes para as funcionalidades e fluxos de adicionar e remover itens ao cardápio, e aos pedidos, além de verificar o valor da taxa de entrega.
- **PedidoTest:** Testes para fazer pedidos imediatos e agendados.
- **RestaurantTest:** Testes relacionados as avaliações dos restaurantes.
- **EntregadorServiceTest:** Testes pensando em relação aos requisitos de cadastro de entregador.
- **RestauranteServiceTest:** Testes pensando em relação aos requisitos de cadastro do restaurante.
- **TestCorridaRepositoryCompleto:** Testes de funcionalidades alteradas e adaptadas: `findByCategoria` e `deleteById`.
- **Testes de ponta a ponta:** Além disso, foram ajustados testes de ponta a ponta, com o objetivo de simular situações de um usuário real utilizando várias funcionalidades em sequência.

6 Considerações Finais

A Sprint 5 representou um marco evolutivo significativo para o sistema UberPB, consolidando com sucesso a sua expansão para o domínio de delivery (modelo UberEats). A iteração

atingiu seu objetivo principal ao entregar um fluxo de pedidos de ponta a ponta, integrando perfeitamente os novos atores (Restaurantes e Entregadores) ao ecossistema já existente.

As implementações realizadas pela equipe garantiram que todo o ciclo de vida do delivery fosse coberto, abrangendo:

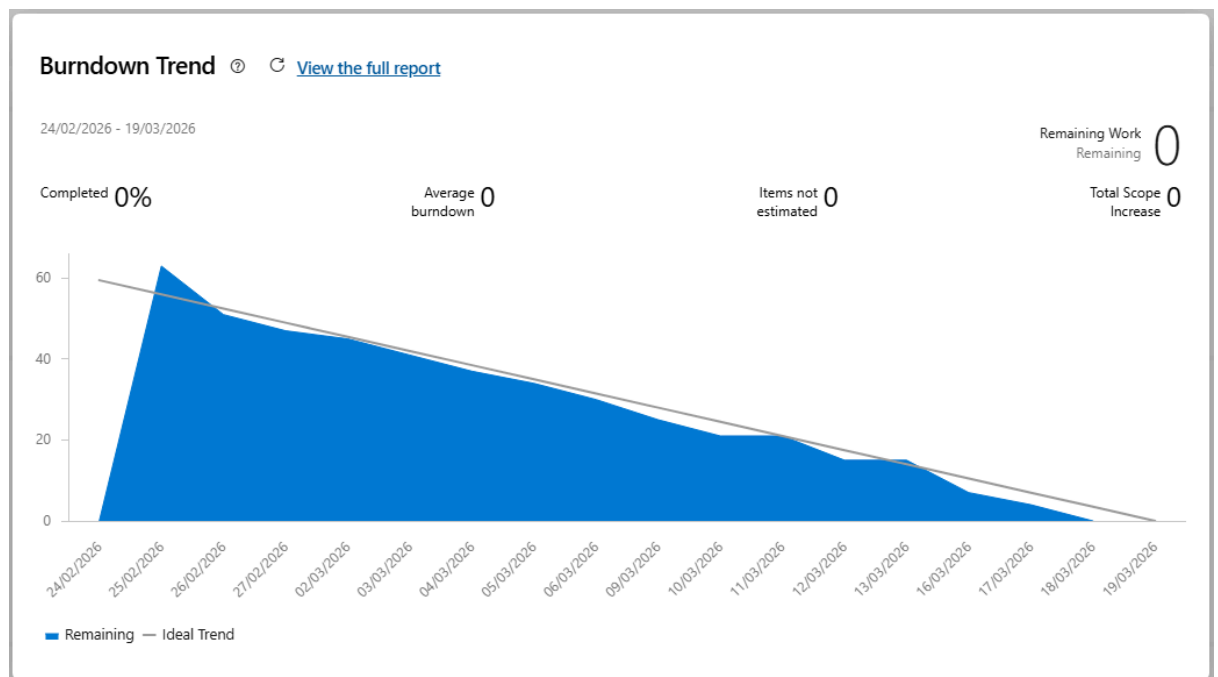
- Entrada no Sistema: Cadastro seguro e validado de restaurantes e entregadores, respeitando regras de negócio específicas (validação de CNPJ e categorias de veículos).
- Gestão de Oferta: Implementação da listagem de estabelecimentos abertos e modelagem robusta de cardápios dinâmicos.
- Ciclo do Pedido: Criação de um sistema de controle de estados eficiente (desde a solicitação, passando pelo preparo, até a entrega), suportando tanto pedidos imediatos quanto agendados.
- Atribuição e Despacho: Lógica estruturada para que entregadores disponíveis assumam demandas de forma organizada.

Do ponto de vista técnico, a manutenção rigorosa da arquitetura em camadas (Model, Service, Repository e CLI) e a aderência a princípios de orientação a objetos (como herança na classe User) foram fundamentais. Essa disciplina arquitetural permitiu que o novo módulo fosse acoplado sem gerar regressões ou comprometer a estabilidade do módulo de mobilidade original. Além disso, a persistência baseada em JSON provou-se flexível o suficiente para acomodar as novas entidades com segurança através do uso de concorrência controlada (Thread-Safety).

Por fim, a elaboração de uma suíte de testes unitários dedicada aos novos requisitos estruturais valida o compromisso da equipe com a qualidade e confiabilidade do software, assegurando que as transições de status e os cálculos de valores ocorram conforme o esperado. O sistema encerra esta Release maduro, funcional e preparado para futuras otimizações de usabilidade e performance.

SPRINT 6

GRÁFICO



RESUMO

A Sprint 6 tem como foco a consolidação e o refinamento da experiência de delivery (modelo UberEats) no sistema UberPB. O objetivo desta iteração é aprimorar a interatividade e o acompanhamento em tempo real do ecossistema de pedidos, englobando a autonomia de aceite e recusa por parte de entregadores e restaurantes (RF24 e RF25), o rastreamento detalhado de status e rotas para clientes e entregadores (RF26 e RF27), a implementação de cálculos avançados de precificação com tarifa dinâmica (RF14) e, por fim, a construção de um sistema de avaliações mútuas que garante a qualidade do serviço entre clientes, entregadores e estabelecimentos (RF28)

ORGANIZAÇÃO DA EQUIPE

- **Manoel Barros (Desenvolvedor):** Responsável por projetar e implementar o sistema de precificação inteligente e tarifa dinâmica do Uber Eats. Atuou ponta a ponta na funcionalidade: desde a modelagem de novos atributos na entidade de domínio, passando pela construção do algoritmo probabilístico de alta demanda, até a integração transparente e detalhada do recibo final na interface do terminal. Ademais, encarregado da documentação geral do software.
- **Arthur Marques (Desenvolvedor):**
- **Guilherme Silva (Desenvolvedor):**
- **Luiz Duarte (Desenvolvedor e Tester):** Responsável pela funcionalidade de avaliação mútua entre restaurante, entregador e usuário consumidor do Uber Eats. Aplicando as funcionalidades e ajustando junto com as outras funcionalidades, e responsável pelos testes utilizando o JUnit. Os testes foram realizados de forma a cobrir as funcionalidades implementadas na Sprint e os testes de ponta-a-ponta simulando o fluxo do sistema.

IMPLEMENTAÇÕES

RF14 – Cálculo Automático e Tarifa Dinâmica no Uber Eats

Responsável: Manoel Barros

Status: Concluído

Para complementar o fluxo de delivery da Sprint 5, foi implementado o requisito RF14, que exige o cálculo automático do valor final do pedido considerando três variáveis: o subtotal dos itens, a taxa de entrega base e a aplicação de tarifa dinâmica para o contexto do Uber Eats.

1.1. Modelagem de Dados (Camada Model) A classe `Pedido` foi expandida para suportar a nova precificação, garantindo o encapsulamento das regras matemáticas:

- **Novos Atributos:** Adição da variável `tarifaDinamica` (inicializada com o multiplicador padrão de 1.0).
- **Cálculo de Totalização:** O método `calcularTotal()` foi refatorado. Agora, ele não apenas soma o subtotal dos objetos `ItemPedido`, mas também aplica o multiplicador da

`tarifaDinamica` sobre a `taxaEntrega` antes de fechar o `valorTotal` do pedido, mantendo a responsabilidade do cálculo dentro da própria entidade.

1.2. Regras de Negócio (Camada Services) Para isolar a lógica de precificação e simulação de demanda da interface visual, as regras foram integradas ao serviço já existente de estimativas:

- **Classe `EstimativaService`:** Criação do método `calcularTarifaDinamicaDelivery()`. Como o sistema opera em ambiente CLI sem APIs externas de trânsito, foi implementado um algoritmo probabilístico que simula alta demanda na região (chuva, horário de pico, escassez de entregadores). O algoritmo possui 30% de chance de ativar a tarifa dinâmica, gerando um multiplicador flutuante e arredondado entre 1.1x e 1.5x.

1.3. Persistência de Dados Devido à arquitetura robusta baseada na biblioteca Jackson (JSON), a adição do novo atributo `tarifaDinamica` na classe `Pedido` foi automaticamente absorvida pelo `PedidoRepositoryJSON`. Os novos pedidos gerados com alta demanda já têm seu multiplicador e valor final salvos no disco sem a necessidade de alterar a camada de repositórios, garantindo consistência na desserialização.

1.4. Interface com o Usuário (Camada CLI) A transparência com o cliente no momento do checkout foi priorizada na atualização da interface:

- **Classe `PedidoCLI`:** O fluxo de finalização de pedido foi atualizado para consultar o `EstimativaService` antes da confirmação.
- **Feedback Visual:** Caso a tarifa dinâmica seja ativada (multiplicador > 1.0), o sistema exibe um alerta visual (" TARIFA DINÂMICA ATIVA") e desmembra o recibo prévio, mostrando a taxa de entrega base e a nova taxa com a dinâmica aplicada. Isso atende ao requisito de clareza do aplicativo sem interromper o fluxo de navegação construído na RF19 e RF20.

RF24 – Aceitação ou Recusa de Pedidos de Entrega pelo Entregador

Responsável: Arthur Marques Araújo

Status: Concluído

1. Descrição da Funcionalidade

A funcionalidade permite que os entregadores visualizem os pedidos disponíveis no sistema e decidam se desejam **aceitar ou recusar cada entrega**.

Esse mecanismo proporciona maior flexibilidade operacional, permitindo que os entregadores gerenciem sua jornada de trabalho de forma mais eficiente. Além disso, garante que pedidos recusados continuem disponíveis para outros entregadores, evitando bloqueios no fluxo de entregas.

2. Implementação

2.1 Modelagem de Dados (Camada Model)

A classe **Pedido** já possuía os atributos necessários para suportar essa funcionalidade:

- status (do tipo StatusPedido)
- entregadorId

RF25 – Confirmação ou Rejeição de Pedidos pelo Restaurante

Responsável: Arthur Marques Araújo

Status: Concluído

1. Descrição da Funcionalidade

Esta funcionalidade permite que os restaurantes **confirmem (aceitem) ou rejeitem (cancelem) pedidos recebidos** por meio do sistema.

Esse mecanismo garante maior controle sobre a operação do restaurante, permitindo que apenas pedidos que possam ser atendidos sejam processados. Dessa forma, contribui para a **organização do fluxo de pedidos e para a manutenção da qualidade do serviço prestado aos clientes**.

2. Implementação

2.1 Modelagem de Dados (Camada Model)

A classe **Pedido** já possuía os estados necessários para representar o fluxo dessa funcionalidade:

- AGUARDANDO_RESTAURANTE
- EM_PREPARO
- CANCELADO

Assim, **não foi necessária nenhuma alteração estrutural na modelagem de dados**, pois os status existentes já contemplavam os cenários de confirmação e rejeição de pedidos.

2.2 Lógica de Negócio (Camada CLI e Repository)

No menu destinado aos restaurantes (**MenuRestauranteCLI**), foi implementada a funcionalidade de **gerenciamento de pedidos recebidos**.

O restaurante pode:

- **Visualizar todos os pedidos vinculados ao seu estabelecimento.**
- Para pedidos com status **AGUARDANDO_RESTAURANTE**, escolher entre:
 - **Confirmar o pedido**
 - O status do pedido é alterado para **EM_PREPARO**, indicando que o restaurante iniciou o preparo.
 - **Rejeitar o pedido**
 - O status do pedido é alterado para **CANCELADO**, encerrando o fluxo do pedido.

A persistência das alterações é realizada por meio do método **updatePedido** da classe **DatabaseManager**, responsável por atualizar os dados por meio do **repositório de pedidos**.

2.3 Interface com o Usuário (Camada CLI)

A interface de linha de comando foi aprimorada para permitir que os restaurantes:

- Visualizem os pedidos recebidos.
- Identifiquem os pedidos que aguardam confirmação.
- Escolham entre **confirmar ou rejeitar** cada pedido.

Após a execução da ação, o sistema apresenta **mensagens claras de feedback**, informando ao usuário o resultado da operação realizada.

RF26 – Acompanhar Status do Pedido (Cliente)

Responsável: Guilherme Henriques

Status: Concluído

1. Descrição da Funcionalidade

Esta funcionalidade permite que os clientes acompanhem, dentro do sistema, o status atual de seus pedidos realizados no módulo de delivery.

O objetivo é oferecer maior transparência durante o processo de entrega, permitindo que o cliente visualize em qual etapa do fluxo o pedido se encontra, desde a criação do pedido até a finalização da entrega.

Com isso, o cliente passa a ter uma melhor percepção do andamento do pedido, reduzindo incertezas durante o tempo de espera e proporcionando uma experiência mais clara durante o processo de delivery.

2. Implementação

2.1 Lógica de Negócio (Camada CLI)

A funcionalidade foi implementada por meio de alterações no menu destinado ao cliente.

Foram realizadas adaptações no **MenuClienteCLI**, permitindo que o usuário consulte seus pedidos ativos e visualize o status atual de cada um deles.

O sistema exibe o estado do pedido de acordo com os valores do atributo **status** presente na classe **Pedido**, que representa as diferentes etapas do fluxo de entrega:

- CRIADO
- AGUARDANDO_RESTAURANTE
- EM_PREPARO
- AGUARDANDO_ENTREGADOR
- EM_ENTREGA
- ENTREGUE
- CANCELADO

Como esses estados já estavam definidos na modelagem da classe **Pedido**, não foi necessária nenhuma alteração estrutural na camada de dados, apenas a implementação da consulta e exibição dessas informações na interface.

2.2 Interface com o Usuário (Camada CLI)

Na interface de linha de comando, o cliente pode acessar a opção de visualizar seus pedidos realizados e acompanhar o status atualizado de cada um deles.

O sistema apresenta mensagens claras indicando o estágio atual do pedido, permitindo que o cliente acompanhe todo o progresso da entrega até sua conclusão.

RF27 – Visualizar Rota até Restaurante e Cliente (Entregador)

Responsável: Guilherme Henriques

Status: Concluído

1. Descrição da Funcionalidade

Esta funcionalidade permite que o entregador visualize a rota simplificada da entrega dentro do sistema, indicando o trajeto entre sua localização atual, o restaurante responsável pelo pedido e o cliente final.

Esse recurso auxilia o entregador na compreensão do fluxo da entrega, apresentando de forma clara os pontos envolvidos no percurso.

A visualização da rota contribui para melhorar a organização do processo de entrega e facilitar o entendimento do trajeto que deverá ser realizado pelo entregador.

2. Implementação

2.1 Interface com o Usuário (Camada CLI)

A funcionalidade foi implementada por meio de alterações na classe **MenuEntregadorCLI**, responsável pelas interações do entregador no sistema.

Durante a visualização de um pedido em andamento, o sistema consulta as informações do restaurante e do cliente associadas ao pedido utilizando o **DatabaseManager**, recuperando seus respectivos endereços e localizações.

Com base nessas informações, o sistema exibe uma representação textual da rota da entrega na interface de linha de comando, indicando:

- Localização atual do entregador
- Endereço do restaurante

- Localização do cliente

Essa visualização é apresentada de forma sequencial na interface, simulando o trajeto da entrega e permitindo que o entregador compreenda facilmente o fluxo do percurso.

2.2 Fluxo de Entrega

A visualização da rota ocorre durante o processo de entrega do pedido, normalmente quando o status já se encontra em **EM_ENTREGA**.

Após visualizar as informações do trajeto, o entregador pode prosseguir normalmente com o fluxo da entrega, incluindo a opção de **finalizar a entrega**, o que atualiza o status do pedido para **ENTREGUE** no sistema.

Essa funcionalidade foi integrada ao fluxo já existente de gerenciamento de entregas, garantindo consistência com os demais requisitos do sistema.

RF28 – O sistema deve permitir que clientes, entregadores e restaurantes se avaliem mutuamente.

4. Permitir que clientes, entregadores e restaurante se avaliem mutuamente

Responsável: Luiz José

Status: Concluído

Foi implementado os tipos de avaliações para os diferentes usuários, possibilitando avaliação mútua entre clientes, entregadores e restaurante, seguindo as especificações do RF28.

4.1 Interface com o Usuário (Camada CLI)

Foram adaptadas as classes:

- MenuEntregador: Implementada a funcionalidade de indisponibilidade do entregador.
- MenuRestaurante: Mudança nos status após o pedido ser aceito, agora o entregador é chamado automaticamente e ele procura entregadores disponíveis.
- PedidoCLI: Ajustada a funcionalidade de tarifa dinâmica para bater com os testes e as avaliações e do recibo do delivery.
 - Mudança dos valores de tarifas com relação aos horários de pico/lotação maior.
 - Aplicação dos valores atualizados, com base nas tarifas aplicadas.
- MenuEntregadorCLI: Implementação da avaliação mútua entre restaurantes, cliente e entregadores.
 - É possível, o entregador, restaurante e os motoristas avaliarem mutuamente os outros.
 - As avaliações acumuladas fazem uma média das avaliações em relação aquele usuário ou restaurante.
 - Atualização automática da média das avaliações.

4.2 Modelagem de Dados armazenados (Data)

Foi adaptadas a classe:

- Restaurantes.json: Adicionado o campo de avaliação.
- Passageiros.json: Adicionado o campo de avaliação.

- Motoristas.json: Adicionado o campo de avaliação.

Com o objetivo de ajustar os dados para avaliações e condicionar os parâmetros das classes.

Testes Funcionais

Responsável: Luiz José

Status: Concluído

Foi implementado os testes referentes ao requisitos implementados para a Sprint 6, RF14, e RF 24 até RF28, utilizando JUnit. No final, totalizando 122 testes de casos de todos os requisitos funcionais do sistema.

5.1 Testes (Camada Test)

Foram criados testes para os requisitos da Sprint, e testes para completar a cobertura do sistema como um todo:

- AvaliacaoETarifaDinamicaTest: implementação de testes de avaliação, tarifas dinâmicas e cálculo/recálculo de média de avaliações.
- PedidoAgendamentoECalculoTest: implementação de testes de cálculo de itens no carrinho, e agendamentos de pedidos, e simulação de pedido agendado.
- FluxoEntregadorIntegrationTest: implementação de testes do aceitando pedido e ficando indisponível, entregador rejeita pedido e passa para o próximo entregador da fila, entregador finaliza a entrega e atualiza o status, e não permitir entregador pegar mais de um pedido de uma vez.